Bounded-Wait Wake Defects in Portable Event Loops

Alex I. Tsvetanov
Faculty of Computer Systems and Technologies
Technical University of Sofia
Sofia, Bulgaria
atsvetanov@tu-sofia.bg

Abstract

An idle event-loop worker parks in a wait with a timeout. When another thread posts a task, the worker must be woken. If the wake is missing, or signals an object the wait does not watch, the task still runs, but only when the timeout expires, so tests of eventual delivery pass. Such missed wakes have been reported in widely used event loops and runtimes. We describe the defect for epoll, io_uring and I/O completion ports, model the delay it causes, and propose a portable test that detects it from latency alone. We measure both on a minimal event loop written for this study, whose correct and defective builds differ in one compile-time switch that guards one site per backend. On Linux the median absolute error of the model is at most 0.53 % of the wait period. On Windows the model holds with the default timer and fails for a 1 ms bound under a 1 ms timer. At a fixed gap the test flagged 10 of 10 defective and 0 of 10 correct configurations, but a gap just below a multiple of the wait period hides the defect. With random gaps, in an exploratory run, it missed 4 of 4000 defective trials and flagged 0 of 4000 correct ones. With a correct wake, the delay on an idle Linux core still depends on the wait bound, also under a 0 μs CPU latency request. Separately, one missed wake in Tokio hangs at a buggy commit and completes at the commit that contains the fix. That case is a hang, and it is not part of the latency matrix.

Keywords: event loop, lost wake-up, epoll, io_uring, I/O completion ports, timer resolution, processor idle states, performance testing.

Status of this draft

This manuscript is a draft for *Software: Practice and Experience*. It is not a submission. The conference running header and the conference front matter are gone. One historical missed wake, in Tokio, is reproduced at the buggy commit by the test under `historical/tokio-yield-lost-wake`, and Section 6.1 states the outcome. That case hangs, because the runtime parks without a bound. The latency matrix was not applied to it, and this draft reports no power and no false-positive rate for it. The matrix itself is one compile-time seed on a minimal loop written for this study. It is not a historical result, and it is not a set of six historical faults. A submission still needs the latency test on a historical bounded-wait miss. The reports cited in the introduction are not that test.

1 Introduction

Servers that multiplex many connections over a few threads run an event loop on each worker thread. A worker waits for I/O events and, when nothing arrives, sleeps in the kernel. Work also comes from other threads: a timer fires, another worker hands over a task, or the application posts a callback. The posting thread must then interrupt the wait.

Many loops bound the wait with a timeout, to check a stop flag or to service timers. The bound turns a missing wake-up into a delay instead of a hang: the task runs when the wait returns on its own. A functional test that checks that a posted task eventually runs therefore passes. The defect shows as latency, and mainly on an idle worker, because a busy one leaves its wait for other reasons and picks the task up on the way.

Studies of real bugs suggest why such defects occur and go unnoticed. In a study of concurrency bugs, atomicity and order violations make up almost all non-deadlock bugs, and one example has the shape of a lost wake-up: a completion callback clears a pending flag before the waiter sets it, and the waiter hangs [1]. In a study of timeout bugs in cloud server systems, most produced no error message, and some degraded performance, for example through idle time behind a timeout that was too long [2]. Missed wake-ups of event

loops occur in widely used software. In Netty, a task submitted from another thread could skip the selector wake-up and wait until the select timeout expired [3]. A race between `epoll_wait` and `epoll_ctl` in Linux lost the `eventfd` event with which Boost.Asio wakes its loop for a handler posted from another thread; in the reporter's test program, `epoll_wait` returned only at its timeout [4-6]. Related defects caused hangs. In libevent, the loop was not woken when another thread deleted its last pending event [7]. A lost wake-up in the Tokio scheduler let a worker park while a spawned task was queued [8], and the .NET thread pool could leave a queued work item unprocessed unless more work was queued [9]. Conversely, Asio, when built for and running on Windows versions before Vista, bounds each wait for completions, because the call can appear stuck while completions are queued [10]: a bounded wait as a safety net.

These are citations of reports. Citing them does not reproduce a bug at its commit. Section 6.1 reproduces one later missed wake in Tokio, a hang in `yield_now` under `block_in_place`, which is not the scheduler hang cited above. The latency matrix in this draft is not that reproduction.

We call this a *bounded-wait wake defect*. We describe how it arises with the kernel interfaces used by portable C++ servers (Section 2), model its delay (Section 3), and propose a portable test that needs no access to the internals of the loop (Section 4). We measure both on a minimal event loop written for this study, with epoll, io_uring and IOCP backends and a compile-time switch that seeds the defect in each (Sections 5 and 6). The repository of the study, with the loop, its tests, the probe, the run scripts, the raw runs, the sanitizer records and the hypotheses with their revision log, is available at `https://github.com/Alex-Tsvetanov/paper-wake-defect`.

2 Background

epoll. The worker waits on a set of file descriptors in `epoll_wait`. The usual cross-thread wake is an `eventfd` [11] in the same set: the poster writes to it and the wait returns.

io_uring. The worker waits for completions in `io_uring_enter` [12]. A write to an `eventfd` ends the wait only if the ring watches that descriptor, for example with a pending `IORING_OP_POLL_ADD`.

I/O completion ports. On Windows the worker waits in `GetQueuedCompletionStatus`, and another thread wakes it by queueing a packet with `PostQueuedCompletionStatus` [13].

A portable loop wraps these behind one post operation. It must ensure that the wait observes every posted task, even when several posts share one signal. The defect class is a failure of that guarantee.

3 Delay model

Consider one worker that alternates between a wait bounded by B and running the tasks it finds. While idle, it returns from the wait with a period $B_{\text{eff}} \geq B$. On Linux, B_{eff} exceeds B by a small measured amount, and the timer slack of the thread can add to it [14]. On Windows, the wait ends on a tick of the timer resolution in effect for the process [15], so B_{eff} can be much larger than B. Let the phase ϕ of a post be the time since the worker last parked. If the post does not end the wait, the task runs at the next timed return, after

$$d(\phi) = B_{\text{eff}} - (\phi \bmod B_{\text{eff}}). \tag{1}$$

With a working wake, the delay is the cost of waking a parked thread and does not follow (1). At uniformly random phase, the defect adds half a period on average to the latency of a task posted to an idle worker. Where timers are routed through the post path, every timer is delayed the same way.

4 Detection test

The test needs only the public post operation, a clock and the configured bound. It lets a one-worker loop go idle, then posts $N = 21$ tasks, each after the previous one has run and a gap has passed. It fails when the median delay exceeds $B_{\text{eff}}/5$. Waiting for each task keeps the worker idle, so that no other event wakes it and hides the defect. The median ignores occasional slow posts, such as one that wakes a core from a deeper idle state [16], and scheduler noise.

The gap sets the phase. With a fixed gap all posts share one phase, and a gap that ends in the last $B_{\text{eff}}/5$ of a period lets a defective loop answer within $B_{\text{eff}}/5$: the test misses. With a random gap the phase is close to uniform, and a defective loop is missed only if more than half of the posts land in that part of the period, which for $N = 21$ has probability $0.097\,\%$. B_{eff} is known only from a defective build; a test of an unknown build can use $B/5$, which is no larger.

Gait measured a probe effect in concurrent programs: a delay inserted into the program can mask synchronization errors [17]. A bounded wait is a delay of that kind. It turns a missed wake into latency, and a test of eventual delivery passes.

The test is a latency oracle. For an omitted or misdirected wake, as in our seeded builds, race detectors have nothing to find: no access is racy, and ThreadSanitizer [18] reports nothing on our defective builds. Schedule exploration in the style of CHESS [19] would not report it either, since every schedule eventually delivers the task. To measure the power of the test we seed the defect with one compile-time switch that guards one site per backend, where it removes or misdirects the wake. This is fault seeding, as in mutation testing [20], with hand-written faults. Related work on wake-up cost in Linux event notification [21], on the exit latency of idle states [16] and on tail latency from power saving [22] measures the cost of a wake that happens; our test detects one that does not.

5 Method

Loop and arms. The loop has one worker thread and no I/O, timers or second worker. A post pushes its task onto a lock-free stack with a compare-and-swap and wakes the worker only if the stack was empty. After every return from its wait, the worker takes the whole stack with one exchange and runs the tasks in post order. The epoll backend writes an `eventfd` registered edge-triggered and waits in `epoll_pwait2` with the bound B. The io_uring backend writes an `eventfd` that a multishot poll in the worker's ring watches, and waits in `io_uring_enter` with a timeout argument. The IOCP backend posts a completion packet and waits in `GetQueuedCompletionStatus` with B rounded up to whole milliseconds. B is a run-time parameter, and the wait can also block. Stop has its own path in every backend. The defect switch guards one site per backend, and a test checks that there is no other: epoll and IOCP omit the wake, and io_uring writes its `eventfd` but never arms the poll that watches it. This is the only seeded defect in the measured loop. It is not a set of six historical faults.

Probe. A standalone program starts the loop, measures B_{eff} in the defective build from 11 calibration posts, and after a few warm-up posts records the phase, delay and prediction of each post. Each post's residual is taken against the B_{eff} of its own run. The gap sweep runs the defective build at $B = 1$ and 10 ms, with 30 runs of 21 posts per configuration, at gaps of $0.1\,B$ to $3.0\,B$ on Linux and target phases of $0.05\,B_{\text{eff}}$ to $2.95\,B_{\text{eff}}$ on Windows. The bound sweep runs the correct build with 51 posts at a 2 ms gap, for $B \in \{1, 10, 100\}$ ms and a blocking wait; on Linux it is repeated under a CPU latency request of 0 μs on `/dev/cpu_dma_latency` [23]. The detection matrix runs both builds at $B = 1$ and 100 ms, with 51 posts at a gap of $1.7\,B$, whose first 21 form the test. An exploratory random-gap design on Linux runs both builds of epoll and io_uring at $B = 1$ and 10 ms, 1000 trials of 21 posts per cell, with gaps uniform in $[0.1\,B, 3.0\,B]$ and a recorded seed. Windows has no such run; for both systems we instead resample the gap sweep, 100000 draws of 21 posts without replacement per configuration, with a recorded seed.

Hypotheses. The hypotheses and their acceptance criteria were fixed before any publication run (Table 1), as their dated revision log in the repository records. Revisions after smoke runs, before any publication run, renumbered and reworded them as H1 to H4 (a prediction from the measured phase and B_{eff}, a fixed-gap test of independence from B, a threshold of $B_{\text{eff}}/5$ instead of $B/5$, an explicit throughput criterion). They also added the exploratory H5 after the smoke data contradicted H2, which was kept as written. The first version's bound of 100 μs on the correct loop's median and its threshold $B/5$ would change no verdict (largest correct median 48.3 μs; the same H3 verdicts). H4 was then deferred and not run for this paper, and we make no claim about it. The hypotheses were first measured on an earlier implementation, which we withdrew; none of its data is used. After those runs and a review, the random-gap design and the resampling of the gap sweep were added as exploratory. Before any sanitizer record or run on the present loop, a last revision made the present loop the measured implementation: the defect is the loop built with the switch on, the fix is the loop with its wake compiled in, and hypotheses, thresholds and designs are unchanged.

Machines. Linux runs use a Ryzen 7 5800H laptop (boost off, performance governor, kernel 7.2.3) with the acpi_idle idle driver and the menu governor [23]; its deepest idle state, C3, reports an exit latency of 350 μs. Windows runs use a Ryzen 5 3600 desktop with Windows 11 and the power plan recorded with the run, whose minimum processor state on mains power is set to its highest value (checked on the host after the runs). The runs measure the loop at commit `47f14f0`, built with clang 22.1.8 and clang-cl 22.1.0; the compiled files' hashes, recorded with each run, match that commit. The timer slack is 1 ns, except in the $B = 1$ ms matrix cells repeated at the default of 50 μs. Windows runs use the default timer resolution or 1 ms from `timeBeginPeriod` (throttling opted out) and time gaps with a high-resolution waitable timer. The cell order is shuffled with a recorded seed, and the Windows run starts only when the processor is on average at least 95 % idle.

Validity. A run counts only if every first-party file compiled into its binaries also passed the loop's tests and

Table 1: Pre-specified hypotheses and their outcomes

	Criterion	Outcome
H1	Defective loop: median |residual| of (1) below 5 % of B_{eff}	Pass on Linux (largest 0.53 %) and on Windows with the default timer (largest 2.73 %); fail on Windows with the 1 ms timer at $B = 1$ ms (32.90 % in the sweep, 7.67 % in the matrix)
H2	Correct loop: largest to smallest median over B at most 1.25	Fail on Linux (epoll 5.357, io_uring 9.409); pass on Windows (1.141 with the 1 ms timer, 1.232 with the default)
H3	The 21-post test flags every defective and no correct configuration	Pass at a gap of $1.7\,B$: 10 of 10 defective and 0 of 10 correct flagged
H4	No measurable throughput cost under load	Deferred, not run; no claim
H5	Exploratory: a 0 μs latency request brings the H2 ratio to 1.25 or below	Fail for epoll (1.924) and io_uring (1.795)

a reduced run of every design, with the same configuration and compiler. The sanitizers were AddressSanitizer with UndefinedBehaviorSanitizer, ThreadSanitizer and MemorySanitizer on Linux, and AddressSanitizer with clang-cl on Windows. None of them reported anything. On both systems the hashes of the compiled files are checked against these records before any cell runs. These records, and those made later for the published commit, are in the repository. The tests pin the loop's semantics in both builds and include wake detectors, which pass in every correct build and detect the seeded defect in every defective build.

6 Results

Delay model. Fig. 1 shows delay against phase at $B = 1$ ms. On Linux the defective loops follow (1): the median absolute residual is at most 0.53 % of B_{eff} over the gap sweeps and matrix cells, and 0.50 % at random phases; at $B = 100$ ms, here and on Windows, it is checked at one phase only. B_{eff} exceeds B by 7.8 μs (epoll) and 6.8 μs (io_uring) at 1 ms, by 332.2 and 323.2 μs at 10 ms, and by 401.4 and 324.0 μs at 100 ms. The default timer slack adds 49.1 μs to the epoll period and leaves io_uring at 1007.0 μs; the man page lists `epoll_wait`, not `io_uring_enter`, among the calls that slack affects [14].

On Windows the wait ends on a timer tick. At the default resolution, bounds of 1 and 10 ms both give one period of 15.7 ms, and 100 ms gives 109709.7 μs, 135.9 μs more than 7 times that measured tick. The largest residual at this resolution is 2.73 %, so H1 holds with the default timer, and the largest delay of a defective loop at bounds of 1 and 10 ms is 16.4 ms. With 1 ms requested, bounds of 10 and 100 ms give periods 93.2 and 273.1 μs longer than B, with residuals of 4.54 and 0.37 %. A 1 ms bound with the 1 ms timer gives a median period of 1519.7 μs. The sweep then loses control of the phase: 502 of 630 posts land within 60 μs of a multiple of 500 μs (24 % if spread evenly), and 405 are delivered after 900 to 1100 μs whatever their phase. There the model misses by 32.90 % of B_{eff} (the matrix cell, at one phase, by 7.67 %), a failure we do not explain, and H1 fails for the 1 ms timer.

Detection. In the matrix the test flagged every defective configuration and passed every correct one (6 per build on Linux, 4 on Windows). Defective medians exceed $B_{\text{eff}}/5$ by factors of 1.24 to 4.34, and the largest correct median is 0.03 of it. $B/5$ gives the same verdicts (factors 1.25 to 69.3). But the matrix has one gap per configuration. The gap sweep runs the same protocol at 30 gaps, and there the test misses in 41 of 240 runs (31 with $B/5$), 37 of them with phases in the last $B_{\text{eff}}/5$ of the period. For epoll at $B = 10$ ms and a 20.0 ms gap, the defective median is 39.9 μs, close to the 36.0 μs of the correct loop at that bound. Random phases shrink this blind spot. Drawing 21 posts at random from each sweep, the largest miss rate is 0.08 % of draws (0.08 % with $B/5$), near the 0.097 % of Section 4. In the exploratory random-gap run on Linux the test missed 4 of 4000 defective trials (0.10 %; 3 with $B/5$) and flagged 0 of 4000 correct ones. For Windows, only the resampling is available.

6.1 A historical missed wake

The citations in the introduction are reports. Separate from those citations, the test under `historical/tokio-yield-lost-wake` builds Tokio at the buggy commit named with the yield deferral report [24] and at the commit that contains the fix [25]. The program parks a worker with `block_in_place`, then waits

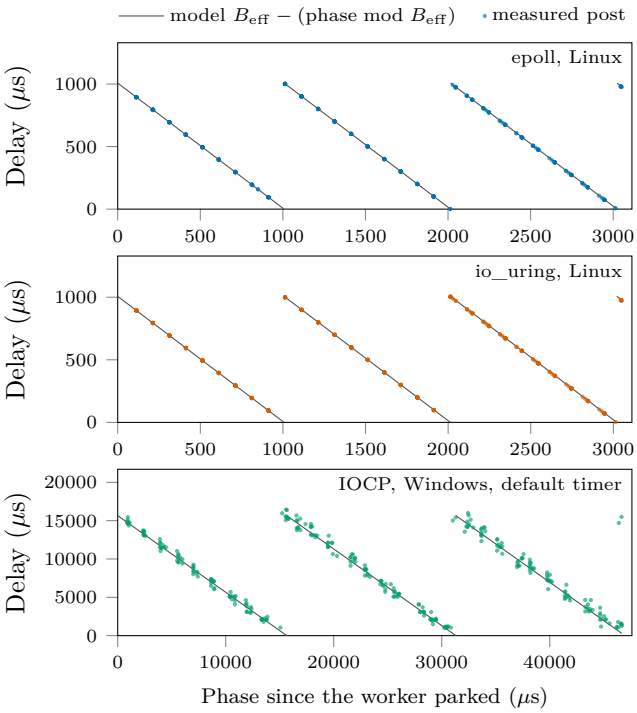


Figure 1: Delay of posted tasks against their phase in a defective loop with $B = 1$ ms: epoll and io_uring on Linux, IOCP on Windows with the default timer resolution. Every second post of each run is shown (300 of 630 per panel). The line is (1) with the measured B_{eff}.

on `yield_now` from an external `block_on`, which is the shape of the test added with the fix. On the buggy commit the task prints its start line and does not reach the line after the yield before the test is stopped. On the commit that contains the fix, the same source prints that line and then returns. The two builds were made on one Windows host, outside the publication runs, and neither build is a sanitizer-gated measurement of this paper's loop. The runtime parks without a bound, so the miss is a hang rather than the bounded delay of (1). The latency test of Section 4 was not applied, and there is no power and no false-positive rate to report. An earlier shape of the same program, without `block_in_place`, did not finish on either commit, so it does not separate them.

Cost of a correct wake. H2 expected the delay of a correct loop not to depend on B. On Windows this holds, with medians of 7.1 to 8.1 μs with the 1 ms timer and 6.9 to 8.5 μs with the default. On Linux it fails: at a 2 ms gap, epoll delivers in 6.8 μs at $B = 1$ ms but in 36.0 to 36.2 μs with longer bounds or a blocking wait, and io_uring in 5.0 μs against 36.1 to 47.0 μs. Under the 0 μs latency request, which leaves idle cores only a polling state [23], every median is lower: epoll delivers in 2.8 μs at $B = 1$ ms and in 5.1 to 5.3 μs otherwise, and io_uring in 2.9 and 5.1 to 5.2 μs. The dependence on B remains, so the exploratory H5 fails for both backends. With and without the request, the 1 ms bound gives the fastest delivery. We did not record idle-state residency, and the data do not show why the shortest bound is fastest.

Threats to validity. Each system ran on one machine, and the machines differ in processor as well as operating system, so the contrast between Linux and Windows in H2 may come from either. All runs use an idle loop without I/O, where the defect shows. Under load, completions end the wait and hide it, so our delays bound its cost from above and say nothing about how often it occurs in production. Each cell ran once, so the variation between runs is unknown. The loop is ours and minimal, and loops with I/O, timers or several workers may wake differently. The seeded faults are few and hand-written, and kqueue is not measured. The test assumes that a correct wake costs much less than $B/5$, which a short bound, a sanitizer build or a deep idle state can break. The random-gap design and the resampling of the gap sweep were added after a review of results from the earlier implementation and were not pre-specified. The Tokio reproduction is one program on one Windows host, stopped by the tester rather than by a pre-registered latency threshold, and it does not measure how often the hang occurs.

7 Conclusion

A wake that misses a bounded wait does not break functional correctness but costs up to one wait period in latency, so functional tests pass. Its delay is a sawtooth in the phase of the post, except for a 1 ms bound under a 1 ms timer on Windows. A test that posts a few tasks to an idle worker and compares their median delay with $B_{\text{eff}}/5$ flagged every seeded defect in our matrix; a fixed gap can hide the defect, and random gaps shrink that blind spot. On Windows the timer resolution of the process rounds the period up: at the default resolution a defect delayed a task by up to one period of about 15.7 ms at bounds of 1 and 10 ms; the largest delay in our runs was 16.4 ms. With a correct wake, the delay on an idle Linux core still depends on the bound, also under a 0 μs CPU latency request. We recommend running the latency test with random gaps alongside functional tests, and controlling or reporting the idle policy in benchmarks of wake latency. A later missed wake in Tokio hangs at the buggy commit and completes once the fix is present (Section 6.1). That reproduction is not a cell of the latency matrix. The matrix remains a self-seeded defect on the loop written for this study, so this draft is not a submission to *Software: Practice and Experience*.

References

[1] S. Lu, S. Park, E. Seo, and Y. Zhou, "Learning from mistakes: A comprehensive study on real world concurrency bug characteristics," in *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Seattle, WA, USA: ACM, Mar. 2008, pp. 329-339.

[2] T. Dai, J. He, X. Gu, and S. Lu, "Understanding real-world timeout problems in cloud server systems," in *2018 IEEE International Conference on Cloud Engineering (IC2E)*. Orlando, FL, USA: IEEE, Apr. 2018, pp. 1-11.

[3] H. Lee, "NioEventLoop ensures that all submitted tasks are executed immediately," Netty commit f44f3e7, GitHub, May 2016, accessed 2026-09-29. [Online]. Available: https://github.com/netty/netty/commit/f44f3e7926f1676315ae86d0f18bdd9b95681d9f

[4] M. Neunhöffer, "There seems to be an in-kernel race between epoll_wait and epoll_ctl which leads to lost events on an eventfd file descriptor," Linux kernel Bugzilla, bug 205933, Dec. 2019, accessed 2026-09-29. [Online]. Available: https://bugzilla.kernel.org/show_bug.cgi?id=205933

[5] M. Neunhöffer, "epoll_reactor occasionally misses wakeup on Linux $\geq$ 5.3," Asio issue 452, GitHub, Jan. 2020, accessed 2026-09-29. [Online]. Available: https://github.com/chriskohlhoff/asio/issues/452

[6] R. Penyaev, "epoll: fix possible lost wakeup on epoll_ctl() path," Linux kernel commit 1b53734bd0b2, Mar. 2020, accessed 2026-09-29. [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=1b53734bd0b2feed8e7761771b2e76fc9126ea0c

[7] A. Khuzhin, "Notify event base if there are no more events, so it can exit without delay," libevent commit 23c2914, GitHub, Apr. 2018, accessed 2026-09-29. [Online]. Available: https://github.com/libevent/libevent/commit/23c2914f6b430f2c2d74c267c13ffab3dda1b325

[8] C. Lerche, "runtime: fix lost wakeup bug in scheduler," Tokio pull request 1788, GitHub, Nov. 2019, fixes issue 1768. Accessed 2026-09-29. [Online]. Available: https://github.com/tokio-rs/tokio/pull/1788

[9] V. Sadov, "Reliability fixes for the Thread Pool," dotnet/runtime pull request 121887, GitHub, Dec. 2025, accessed 2026-09-29. [Online]. Available: https://github.com/dotnet/runtime/pull/121887

[10] C. M. Kohlhoff, "asio/detail/win_iocp_io_context.hpp," Asio source code, commit 8806a68, lines 274 to 282, Jul. 2026, accessed 2026-09-29. [Online]. Available: https://github.com/chriskohlhoff/asio/blob/8806a6803cde7054c3049d3666d3ec36786568c5/include/asio/detail/win_iocp_io_context.hpp#L274-L282

[11] Linux man-pages project, "eventfd(2): Create a file descriptor for event notification," Linux man-pages 6.19, Feb. 2026, accessed 2026-09-25. [Online]. Available: https://man7.org/linux/man-pages/man2/eventfd.2.html

[12] liburing project, "io_uring_enter(2): Initiate and/or complete asynchronous I/O," liburing manual page, accessed 2026-09-25. [Online]. Available: https://man7.org/linux/man-pages/man2/io_uring_enter.2.html

[13] Microsoft, "I/O completion ports," Win32 documentation, Microsoft Learn, Jul. 2026, accessed 2026-09-25. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/fileio/i-o-completion-ports

[14] Linux man-pages project, "PR_SET_TIMERSLACK(2const): Set the "current" timer slack value for the calling thread," Linux man-pages 6.19, Feb. 2026, accessed 2026-09-26. [Online]. Available: https://man7.org/linux/man-pages/man2/PR_SET_TIMERSLACK.2const.html

[15] Microsoft, "timeBeginPeriod function (timeapi.h)," Win32 documentation, Microsoft Learn, accessed 2026-09-26. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/timeapi/nf-timeapi-timebeginperiod

[16] R. Schöne, D. Molka, and M. Werner, "Wake-up latencies for processor idle states on current x86 processors," *Computer Science - Research and Development*, vol. 30, no. 2, pp. 219-227, May 2015.

[17] J. Gait, "A probe effect in concurrent programs," *Software: Practice and Experience*, vol. 16, no. 3, pp. 225-233, Mar. 1986.

[18] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data race detection in practice," in *Proceedings of the Workshop on Binary Instrumentation and Applications (WBIA)*. New York, NY, USA: ACM, Dec. 2009, pp. 62-71.

[19] M. Musuvathi, S. Qadeer, T. Ball, G. Basler, P. A. Nainar, and I. Neamtiu, "Finding and reproducing heisenbugs in concurrent programs," in *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08)*. San Diego, CA: USENIX Association, Dec. 2008. [Online]. Available: https://www.usenix.org/conference/osdi-08/finding-and-reproducing-heisenbugs-concurrent-programs

[20] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649-678, Sep. 2011.

[21] L. Nolte, T. Twardzik, C. Jalier, Z. Huang, J. Shi, C. Kowalsky, T. Wild, and A. Herkersdorf, "HAWEN: Hardware accelerator for thread wake-ups in Linux event notification," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. San Francisco, CA, USA: IEEE, Jul. 2023, pp. 1-6.

[22] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. Seattle, WA, USA: ACM, Nov. 2014, pp. 1-14.

[23] The Linux Kernel documentation, "CPU idle time management," Admin guide, kernel 7.3.0-rc4, accessed 2026-09-26. [Online]. Available: https://docs.kernel.org/admin-guide/pm/cpuidle.html

[24] quininer, "external executor block_on and yield defer do not work together," Tokio issue 6996, GitHub, Nov. 2024, buggy source reproduced here: Tokio 1.41.1, commit bb7ca7507b94d01ffe0e275ddc669734ab3bf783. Accessed 2026-10-22. [Online]. Available: https://github.com/tokio-rs/tokio/issues/6996

[25] quininer, "runtime: fixed yield_now lost wakeup," Tokio commit c6f9e61, pull request 6999, GitHub, Nov. 2024, contained in Tokio 1.42.0, commit bb9d57017e100985f86d8ca41ac105ee9140423e. Accessed 2026-10-22. [Online]. Available: https://github.com/tokio-rs/tokio/commit/c6f9e61d9ee7e69c1e6c47dacd5a7478bc011b59